

On Tuesday, September 26, 2023, GitLab (git.doit.wisc.edu) will be upgraded from 15.8 to GitLab 16.2. Learn more about the upgrade: <https://kb.wisc.edu/shared-tools/news.php?id=13235>

p

p2

Project ID: 15868



instructions added
LOUIS TYRRELL OLIPHANT authored 6 days ago

Name	Last commit	Last update
README.md	instructions added	6 days ago

[README.md](#)

CS 537 Project 2 -- Creating an xv6 System Call

Administrivia

- **Due Date:** September 26th, at 11:59pm
- Projects may be turned in up to 3 days late but you will receive a penalty of 10 percentage points for every day it is turned in late.
- **Slip Days:**
 - In case you need extra time on projects, you each will have 2 slip days for individual projects and 2 slip days for group projects (4 total slip days for the semester). After the due date we will make a copy of the handin directory for on time grading.
 - To use a slip day you will submit your files with an additional file **slipdays.txt** in your regular project handin directory. This file should include one thing only and that is a single number, which is the number of slip days you want to use (ie. 1 or 2). Each consecutive day we will make a copy of any directories which contain one of these slipdays.txt files.
 - After using up your slip days you can get up to 90% if turned in 1 day late, 80% for 2 days late, and 70% for 3 days late. After 3 days we won't accept submissions.
 - Any exception will need to be requested from the instructors.
 - Example slipdays.txt

1

- Some tests are provided at `~cs537-1/tests/P2` . There is a `README.md` file in that directory which contains the instructions to run the tests. The tests are partially complete and you are encouraged to create more tests.
- Questions: We will be using Piazza for all questions.
- Collaboration: The assignment must be done by yourself. Copying code (from others) is considered cheating. [Read this](#) for more info on what is OK and what is not. Please help us all have a good semester by not doing this.
- This project is to be done on the [Linux lab machines](#), so you can learn more about programming in C on a typical UNIX-based platform (Linux). Your solution will be tested on these machines.
- **Handing it in:** Copy your files to `~cs537-1/handin/login/P2` where login is your CS login.

Project Brief

In this project you will add a new system call to the xv6 operating system. More specifically, you will have to implement a system call named `getlastcat` with the following signature:

```
int getlastcat(char*)
```

The system call takes a character pointer as its argument; it will copy the most recent filename passed as an argument to the `cat` program into the character buffer pointed to by the argument. For example, if the user had used `prompt> cat README` prior to calling `getlastcat` , it will copy `README` to the character buffer. If the user has not yet run `cat` since the OS booted, `getlastcat` should copy `Cat has not yet been called` to the character pointer.

- If cat had been called with more than one argument like `prompt> cat f1 f2 f3`, the last argument i.e `f3` should be copied into the buffer.
- If cat had been called without any arguments (that can be done by `prompt> cat` and then pressing `ctrl+d`), the system call should copy "No args were passed" to the buffer.
- In all of the above cases, the return value of the system call should be 0 to indicate success.
- If a user program calls the system call with a null pointer, it should return -1 to indicate failure.
- If cat was called with a file that did not exist, the system call should copy `Invalid filename` to the buffer. If cat was called with multiple arguments and one of those was non-existent, the system call should still copy `Invalid filename` to the buffer regardless of the position of the non-existent file in the argument list.

Please note that passing parameters to syscalls and returning values from system calls is a bit tricky and does not follow the normal parameters and return semantics that we expect to see in userspace. A good starting place would be to look at how the `chdir` system call is implemented, and how its arguments are handled. Remember that one of the goals of this assignment is to be able to navigate a large codebase and find examples of existing code that you can base your work on.

You will also create a new user level program, also called `getlastcat` which will call your system call and print its output in the following format `XV6_TEST_OUTPUT Last catted filename: syscalloutput` where `syscalloutput` is a placeholder for the output returned by your system call.

Important: You are not allowed to change the implementation of any user level program except `getlastcat`

Objectives:

- Understand the xv6 OS, a simple UNIX operating system.
- Learn to build and customize the xv6 OS
- Understand how system calls work in general
- Understand how to implement a new system call
- Be able to navigate a larger codebase and find the information needed to make changes.

Summary of what gets turned in:

- The entire XV6 directory has to be turned in with modifications made in appropriate places to add the new system call and a new user program. xv6 should compile successfully when you run `make qemu-nox`. **Important: Please make sure that you run `make clean` before submitting the XV6 directory**
- Your project should (hopefully) pass the tests we supply.
- **Your code will be tested in the CSL Linux environment (Ubuntu 22.04.3 LTS). These machines already have qemu installed. Please make sure you test it in the same environment.**
- Include a single README.md describing the implementation in the top level directory. This file should include your name, your cs login, you wisc ID and email, and the status of your implementation. If it all works then just say that. If there are things which don't work, let us know. Please **list the names of all the files you changed in xv6**, with a brief description of what the change was. This will **not** be graded, so do not sweat it.
- The README.md should also contain your LLM conversations. You are allowed to use Large-Language Models like OpenAI's ChatGPT or GitHub's Copilot. As these generative tools are transforming many industries (including computer science and education) I am seeking understanding on how best to incorporate them into the classroom and course projects; however you are required to turn in a document of all interactions about the project with all LLMs. Be aware that when you seek help from the instructional staff, we will not assist with working with these LLMs.

Getting xv6 up and running

The xv6 operating system is present in the `~cs537-1/xv6/` directory on CSL AFS. The directory contains instructions on how to get the operating system up and running in a `README.md` file. The directory also contains the manual for this version of the OS in the file `book-rev11.pdf`.

We encourage you to go through some resources beforehand:

1. [Discussion video](#) - Remzi Arpaci-Dusseau.
2. [Discussion video](#) - Shivaram Venkataraman.
3. [Some background on xv6 syscalls](#) - Remzi Arpaci-Dusseau.

Adding a user-level program to xv6

As an example we will add a new user level program to the xv6 operating system. After untarring XV6 (`tar -xzf ~cs537-1/xv6/xv6.tar.gz -C .`) and changing into the directory, create a new file called `test.c` with the following contents

```
#include "types.h"
#include "stat.h"
#include "user.h"
int main(void)
{
    printf(1, "The process ID is: %d\n", getpid());
}
```

```
    exit();  
}
```

Now we need to edit the Makefile so our program compiles and is included in the file system when we build XV6. Add the name of your program to `UPROGS` in the Makefile, following the format of the other entries.

Run `make qemu-nox` again and then run `test` in the xv6 shell. It should print the PID of the process.

You may want to write some new user-level programs to help test your implementation of `getlastcat`.